

Unsupervised Learning

Everything so far; linear regression, logistic regression, decision trees; have been **supervised** learning methods. We have **labelled** training data.

Unsupervised learning removes these labels entirely. The goal is now to discover **structure** in the data itself, like patterns, groupings or anomalies. In many ways, this is a harder problem, there are no obvious objective functions to minimise, and evaluating whether a solution is good becomes less clear cut.

Clustering Algorithms

Can we divide the data into **groups** such that points within the groups are similar to each other, and different to points outside the group?

K-Means Clustering

Given n unlabelled points $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$ and a chosen number of clusters K , we want to assign each point to **exactly one** cluster.

Now, we want points in the same cluster closer together, and different clusters far apart. Formally, we have K clusters $C_1, \dots, C_K$ and a set of **cluster centers (centroids)** $\mu_1, \dots, \mu_K \in \mathbb{R}^d$ that minimise the total within-cluster sum of squared distances:

$$J = \sum_k \sum_{i \in C_k} \|x^{(i)} - \mu_k\|^2$$

This objective function J is called the **inertia** or **distortion**. It is a measure of how tightly packed each cluster is around its centroid. K-means clustering wants to minimise J .

The Algorithm

We **alternate between 2 steps** until convergence:

- **Assignment**: assign each point to the **nearest centroid**
- **Update**: recompute the centroid as the mean of all points currently assigned to it.

We repeat this until the **assignments no longer change** - Convergence.

Each step **provably decreases or maintains** J , so the algorithm is guaranteed to converge. However, it converges to a **local minimum of J** , not necessarily the global minimum. The solution found depends on the **initial centroid positions**.

It's worth understanding why each step decreases or maintains J , never increasing it.

The Assignment Step

In this step, the centroids μ_k are fixed. We reassign each point to a cluster. For a single point $x^{(i)}$, its contribution to J is $\|x^{(i)} - \mu_{\text{current}}\|^2$ where μ_{current} is whichever centroid it is currently assigned to. We reassign it to C_i by computing

$$C_i = \operatorname{argmin}_k \|x^{(i)} - \mu_k\|^2$$

Now, **argmin** is short for **argument of the minimum**, it returns the **input value** that minimises a function, rather than the minimum value itself.

To give an example, suppose you have a function $f(k)$.

- **min $f(k)$** gives the smallest value f takes.
- **argmin $f(k)$** gives the value of k that produces that smallest value.

So in the k -means context, $\operatorname{argmin}_k \|x^{(i)} - \mu_k\|^2$ returns the index k of the centroid that achieves that smallest distance. That index is the cluster we assign point i to.

So, by definition, the new centroid is at least as close as the old one.

$$\|x^{(i)} - \mu_{\text{new}}\|^2 \leq \|x^{(i)} - \mu_{\text{current}}\|^2$$

So the contribution of this point to J can only decrease or stay the same. Since this is true for all points independently, the total J can only decrease or stay the same.

The update step

Now, the assignments are fixed. How do we optimally recompute the centroids to minimise J ? Let's consider a single cluster k . Its contribution to J is:

$$J_k = \sum_i \in C_k \|x^{(i)} - \mu_k\|^2$$

We want the value of μ_k that minimises this. We differentiate w.r.t μ_k :

$$\frac{dJ_k}{d\mu_k} = \sum_i \in C_k - 2(x^{(i)} - \mu_k)$$

Setting to zero:

$$-2 \sum_i \in C_k (x^{(i)} - \mu_k) = 0$$

$$-2 \sum_i \in C_k (x^{(i)}) + 2i \mu_k = 0$$

$$2i \mu_k = 2 \sum_i \in C_k (x^{(i)})$$

$$\mu_k = \frac{1}{i} \sum_i \in C_k (x^{(i)})$$

$$\therefore \mu_k = \frac{\sum_i x^{(i)}}{i}$$

exactly the mean.

So we've provably chosen the value of μ_k that minimises J_k given the current assignments.

So both steps decrease or maintain J . Since it is strictly decreasing until it stagnates, the algorithm always reaches a point where neither step changes anything - convergence.

K-Means ++

As discussed, K-means converges to a local minimum, dependent on the initial choice of centroids. Random initialisation frequently produces poor solutions. K-Means ++ is a smarter initialisation strategy that dramatically improves results in practice.

The procedure:

- Choose the first centroid uniformly at random from the data points.
- For each subsequent centroid: compute the squared distance from each point to the nearest, already-chosen centroid. Sample the next centroid from the data points with probabilities proportional to those squared distances (So points far from existing centroids are likely to be chosen as new centroids).
- Repeat until k centroids are chosen, then run standard k means.

In practice, always use k -means ++. The computational overhead is negligible and the quality improvement is substantial.

Choosing k

K-means requires you to specify k in advance, which is often unknown. There is no universally correct answer, but there are a few practical methods:

- **Elbow method**

plot J (inertia) as a function of k . As k increases, J decreases monotonically - more clusters always fit the data better. But there is typically a point of **diminishing returns** where the rate of decrease slows sharply, forming an **"elbow"** in the curve. The elbow is a reasonable heuristic for the right k . In practice, the precise location of the elbow is ambiguous.

- **Silhouette Score**

This gives us one number that provides insight into the quality of the clustering. For a single point $x^{(i)}$, we measure two quantities. $a^{(i)}$ - the mean distance from $x^{(i)}$ to all other points in its **own cluster**, and $b^{(i)}$ - the mean distance from $x^{(i)}$ to all points in the **nearest other cluster**.

Small $a^{(i)}$ means it's tightly in its own cluster - good. Large $b^{(i)}$ means it's far from other clusters - good. We compute $s^{(i)}$

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max(a^{(i)}, b^{(i)})}$$

So $s^{(i)} \in [-1, +1]$.

We average $s^{(i)}$ across all clusters to get an overall quality score.

- **Domain knowledge**

In many applications, the number of clusters has a natural interpretation, like number of product categories. We can then choose k from context rather than data.

Limitations of k-Means

K-means makes strong **implicit assumptions** about the structure, that is often violated in practice. The drawbacks include:

- **Assumption of Spherical Clusters**

Since we use radial distance from the centroid, k-means implicitly assumes clusters are spherical. Additionally, since we are minimising within-cluster variance, we are also assuming that all spheres are equally sized. Very large spheres would still be penalised, even though they form a valid cluster. So k-means struggles with **elongated**, **non-convex** or very **differently sized clusters**.

- **Sensitivity to Outliers**

A single outlier far from any cluster can dramatically pull a centroid, distorting the whole solution.

- **Requires feature scaling**

Since k-means uses Euclidean space, features with large magnitudes dominate. Always standardise before running k-means.

- **Hard Assignments**

Every point belongs to **exactly one cluster** with full certainty. Real data often has **ambiguous points** that genuinely could belong to multiple clusters.

- **Must Specify k in advance**

We don't always know this before hand

Gaussian Mixture Models

GMMs solve the problem of hard cluster assignments, they instead provide a **probability** that a data point belongs to each class. First, we need some mathematical foundations.

Covariance

When dealing with one variable, we already know about **variance**, a measure of how **spread out** the data is around the mean.

$$\text{Var}(x) = \frac{1}{n} \sum_i (x_i - \bar{x})^2$$

However, in machine learning, data points have multiple features, and those features often **move together**. For example:

- **Height** and **Weight** tend to increase together - tall people tend to be heavier
- **Hours studied** and **exam scores** tend to increase together.

Variance doesn't capture this relationship. $\text{Var}(\text{Height})$ and $\text{Var}(\text{Weight})$ separately tell you nothing about how they relate to each other. On the other hand, **covariance** measures how two variables vary **together**.

$$\text{Cov}(x, y) = \frac{1}{n} \sum_i (x_i - \bar{x})(y_i - \bar{y})$$

Let's unpack the product $(x_i - \bar{x})(y_i - \bar{y})$.

- If both x and y are above the mean the product is **positive**
- If both x and y are below the mean the product is **positive** ($-ve \times -ve = +ve$)
- If x is above the mean but y is below the mean (or vice versa) the product is **negative** ($+ve \times -ve = -ve$)

So we interpret the covariance from its **sign**.

• **Positive Covariance**

x and y tend to move in the **same direction**

• **Negative Covariance**

x and y tend to move in **opposite directions**

• **Zero Covariance**

x and y are **unrelated**, knowing one tells you nothing about the other.

For d features, we build a **covariance matrix** where $\Sigma_{i,i} = \text{Cov}(x_i, x_i)$. Note that $\text{Cov}(x, x) = \text{Var}(x)$ and $\text{Cov}(x, y) = \text{Cov}(y, x)$. This means the variance lies on the diagonal, and the matrix is symmetric.

Understanding GMMs

GMMs imagine that the data was **generated based on a probability distribution**. More specifically, that it was generated by a **mixture of k Gaussian distributions**, one for each cluster. We have a **weighted sum** of these distributions, where the weight corresponds to the size of the cluster. A large cluster holds a lot of the data points, so when drawing from our mixed distribution, there must have been a higher probability of drawing from that cluster, hence a higher weight. This overall distribution is given by:

$$p(x) = \sum_k \pi_k \cdot N(\mu_k, \Sigma_k)$$

Where:

- π_k is the **mixing weight**
- μ_k is the **mean**
- Σ_k is the **covariance**

Our goal is to learn π_k , μ_k and Σ_k

Given a trained model, a data point $x^{(i)}$ sits somewhere in space. We want to know: which Gaussian most likely generated this point? We want a probability for each cluster k . This is a conditional probability

$$p(z=k | x^{(i)})$$

The probability it belongs to cluster k given its location in space. Applying Bayes Theorem:

$$p(A|B) = \frac{p(B|A) p(A)}{p(B)}$$

$$p(z=k | x^{(i)}) = \frac{p(x^{(i)} | z=k) p(z=k)}{p(x^{(i)})}$$

Let's identify each term:

- $p(z=k)$ is the prior probability of component k , how prominent that cluster is before we look at $x^{(i)}$ at all. This is just the mixing weight π_k .

- $p(x^{(i)} | z=k)$ is the likelihood of observing $x^{(i)}$ if it did come from component k . This is just the Gaussian probability density function evaluated at $x^{(i)}$. $\leftarrow$ (a scalar)

$$N(x^{(i)}; \mu_k, \Sigma_k)$$

- $p(x^{(i)})$ is the total probability of observing $x^{(i)}$ under the full mixture model, as seen before:

$$p(x^{(i)}) = \sum_j \pi_j \cdot N(x^{(i)}; \mu_j, \Sigma_j)$$

Putting this all together:

$$r(i, k) = \frac{\pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k)}{\sum_j \pi_j \cdot N(x^{(i)}; \mu_j, \Sigma_j)}$$

Responsibility combines two things: proximity and size. How close is the point to

each Gaussian centre (weighted by its shape) and how prominent each cluster is.

Training: The Expectation-Maximisation Algorithm

In a similar way to other ML algorithms, let's first consider what happens as we try to maximise the log likelihood.

We have observed data X and want to find parameters $\Theta = \{\mu_k, \Sigma_k, \pi_k\}$ that maximise $\log p(X; \Theta)$.

Recall that:

$$p(x^{(i)}) = \sum_k \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k)$$

Thus, since each data point is independent, the probability of seeing the entire data set is given by the product of the probs of each datapoint:

$\leftarrow$ likelihood

$$\mathcal{L} = P(X; \Theta) = \prod_i \sum_k \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k)$$

Note: Recall we are viewing this through the Likelihood lens, asking ourselves: "Given these parameters, how likely is this observed data set?" We want to find the parameters that maximise the likelihood.

Taking the log:

$$\log \mathcal{L} = \sum_i \log \left(\sum_k \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k) \right)$$

This is a problem, since we still have the log of a sum on the inside. We can't tractably differentiate this to maximise the log likelihood. How would we get rid of that infernal sum? Recall that the inner sum comes from:

$$p(x^{(i)}) = \sum_k \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k)$$

Think for a moment, what is the reason this probability must be a sum? It's because we **don't know which Gaussian generated the data point**. If we knew this information, there would be no need to sum over all of them, we would just pick out the relevant one and compute the probability from there. So, what would the likelihood expression look like if we **knew all the assignments**? We have a random variable Z where each $z^{(i)} = k$ represents the cluster labels for each data point. So now we're maximising a **different likelihood**. Instead of trying to maximise the **observed likelihood** $P(X)$ (probability of the data we can see), we're trying to maximise the **complete likelihood** $P(X \text{ AND } Z)$ (probability of data plus hidden assignments).

By the probability chain rule:

$$P(X, Z) = P(X) \cdot P(Z | X)$$

Then by taking the log of this

$$\begin{aligned} \log(P(X, Z)) &= \log(P(X) \cdot P(Z | X)) \\ &= \log(P(X)) + \log(P(Z | X)) \end{aligned}$$

Consider $P(Z | X)$. We're assuming that Z is known with certainty, which means that $P(Z | X)$ **must be 1**.

$$\begin{aligned} \log(P(X, Z)) &= \log(P(X)) + \log(1) \\ &= \log(P(X)) \end{aligned}$$

So we've just shown that maximising the log likelihood of the **complete data** $P(X, Z)$ is identical to maximising the LL of the **observed data** $P(X)$.

So to step back, we've established that

not knowing Z makes maximising the log likelihood **hard** (we have a sum inside the log). However, if we **do know Z** , the problem becomes easy (no sum):

$$\begin{aligned} P(x^{(i)}, z^{(i)} = k) &= P(z^{(i)} = k) \cdot P(x^{(i)} | z^{(i)} = k) \\ &= \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k) \end{aligned}$$

$$\therefore P(X, Z) = \prod_i \pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k)$$

$$\Rightarrow \log(P(X, Z)) = \sum_i \log(\pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k))$$

No Sum in the log

So the logical next step is to ask: **how can we approximate Z ?** If we have an approximation for Z , we can use it above, and solve our problem. To formalise this, we make use of an **indicator function**

$$1[z^{(i)} = k]$$

This function returns 1 if $z^{(i)} = k$ is true, and 0 if it is false. Then we turn that inner expression into another sum:

$$\log L = \sum_i \sum_k 1[z^{(i)} = k] \log(\pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k))$$

Note that this expression is exactly equivalent to the one above. All terms where $z^{(i)} \neq k$ get zeroed out, leaving only the correct term in its place. We do this so that our question of "How do we approximate Z ?" turns into the concrete: **What is the expected value of $1[z^{(i)} = k]$?**

$$E[1[z^{(i)} = k]] = P(z^{(i)} = k | x^{(i)})$$

The expected value is the probability that $z^{(i)} = k$ given the observed data.

This should be familiar, it's exactly the **responsibility** $r(i,k)$ that we calculated earlier! It acts as our soft approximation to the hard indicator. Instead of saying "point i definitely came from component k ", we say "point i came from component k with probability $r(i,k)$ ".

This is the **E-step** (Expectation step) in the EM-algorithm. We calculate the complete data log-likelihood with soft responsibilities:

$$Q = \sum_i \sum_k r(i,k) \cdot \log(\pi_k \cdot N(x^{(i)}; \mu_k, \Sigma_k))$$

We then **maximize** Q with respect to the parameters, the M-step. We need to differentiate Q w.r.t each parameter.

$$Q = \sum_i \sum_k r(i,k) [\log(\pi_k) + \log(N(x^{(i)}; \mu_k, \Sigma_k))]$$

note that:

$$N = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)\right)$$

$$\text{So } \log(N(x^{(i)}; \mu_k, \Sigma_k))$$

$$= -\log((2\pi)^{n/2} \cdot |\Sigma_k|^{1/2}) - \frac{1}{2} (x^{(i)} - \mu_k)^T \Sigma_k^{-1} (x^{(i)} - \mu_k)$$

$$= -\log((2\pi)^{n/2}) - \frac{1}{2} \log(|\Sigma_k|) - \frac{1}{2} (x^{(i)} - \mu_k)^T \Sigma_k^{-1} (x^{(i)} - \mu_k)$$

Since we're going to be differentiating, we can discard the constant term in the front:

$$= -\frac{1}{2} \log(|\Sigma_k|) - \frac{1}{2} (x^{(i)} - \mu_k)^T \Sigma_k^{-1} (x^{(i)} - \mu_k)$$

So, plugging back into Q :

$$Q = \sum_i \sum_k r(i,k) \left[\log(\pi_k) - \frac{1}{2} \log(|\Sigma_k|) - \frac{1}{2} (x^{(i)} - \mu_k)^T \Sigma_k^{-1} (x^{(i)} - \mu_k) \right]$$

Note that this has cleanly split the 3 parameters into 3 terms that can be

can be considered independently. Let's consider the μ_k term first.

$$Q_{\mu} = \sum_i \sum_k r(i,k) \left[-\frac{1}{2} (x^{(i)} - \mu_k)^T \Sigma_k^{-1} (x^{(i)} - \mu_k) \right]$$

We want to optimise over a specific μ_k in turn, so we consider:

$$\frac{\partial Q}{\partial \mu_j} = \sum_i \frac{\partial}{\partial \mu_j} \left(r(i,j) \left[-\frac{1}{2} (x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j) \right] \right)$$

$$= -\frac{1}{2} \sum_i r(i,j) \frac{\partial}{\partial \mu_j} \left((x^{(i)} - \mu_j)^T \Sigma_j^{-1} (x^{(i)} - \mu_j) \right)$$

Using the identity $\frac{\partial}{\partial \mu} (x-\mu)^T A (x-\mu) = -2A(x-\mu)$ for symmetric A :

$$\frac{\partial Q}{\partial \mu_j} = \sum_i r(i,j) \Sigma_j^{-1} (x^{(i)} - \mu_j)$$

Setting to zero:

$$\sum_i r(i,j) \Sigma_j^{-1} (x^{(i)} - \mu_j) = 0$$

Since Σ_j^{-1} is invertible, we can multiply through by Σ_j

$$\sum_i r(i,j) (x^{(i)} - \mu_j) = 0$$

$$\sum_i r(i,j) x^{(i)} - \sum_i r(i,j) \mu_j = 0$$

renamed back to k

$$\mu_k = \frac{\sum_i r(i,k) x^{(i)}}{\sum_i r(i,k)}$$

Perform a similar process for the mixture weights π_k .

$$Q_{\pi} = \sum_i \sum_k r(i,k) \log(\pi_k)$$

$$\frac{\partial Q_{\pi}}{\partial \pi_j} = \sum_i \frac{\partial}{\partial \pi_j} (r(i,j) \log(\pi_j))$$

But be careful! This is subject to

$$\sum_k \pi_k = 1$$

This is a constrained optimization problem.
We form the Lagrangian:

$$\mathcal{L}(\pi_k, \lambda)$$

$$= \sum_i \sum_j r(i,j) \log(\pi_k) - \lambda \left(\sum_k \pi_k - 1 \right)$$

differentiate w.r.t π_j :

$$\frac{\partial \mathcal{L}}{\partial \pi_j} = \frac{\sum_i r(i,j)}{\pi_j} - \lambda$$

Set to 0:

$$\sum_i r(i,j) = \lambda \pi_j$$

$$\pi_k = \frac{1}{\lambda} \sum_i r(i,k)$$

Use the constraint:

$$\sum_k \pi_k - 1 = 0$$

If we define $N_k = \sum_i r(i,k)$, then:

$$\frac{1}{\lambda} \sum_k N_k = 1$$

$$\text{but } \sum_k N_k = N \text{ (number of data points)}$$

$$\Rightarrow \lambda = N$$

So:

$$\pi_k = \frac{N_k}{N}$$

And finally, for the covariance:

$$Q_{\Sigma} = -\frac{1}{2} \sum_i r(i,k) \left[\log |\Sigma_k| + (x_k - \mu_k)^T \Sigma_k^{-1} (x_k - \mu_k) \right]$$

Taking matrix derivatives and setting to zero gives:

$$\Sigma_k = \frac{\sum_i r(i,k) (x_k^{(i)} - \mu_k)(x_k^{(i)} - \mu_k)^T}{\sum_i r(i,k)}$$

The Full Algorithm

• Initialisation

Choose K and initialise the parameters somehow - typically by running k -means to get sensible starting centroids.

We then set μ_k to the starting centroids, Σ_k to the identity matrix and $\pi_k = 1/K$.

• E-Step

Given the current parameters, compute all the responsibilities $r(i,k)$. We're asking the question: given where each point sits in space, and what each Gaussian currently looks like, how much credit does each component get for having generated each point?

• M-Step

Using the responsibilities as fixed weights, update the parameters to maximise Q . If you look at the expressions for each update rule, you see that they are all **responsibility weighted** statistics. Each point contributes in proportion to how likely it is to belong to that component.

• Check Convergence

Compute the observed log likelihood

$$\log \mathcal{L} = \sum_i \log \left[\sum_k \pi_k \cdot N(x_k^{(i)}; \mu_k, \Sigma_k) \right]$$

If the change since the last iteration is below some threshold, stop. Otherwise repeat again from the E-step.

Like k -means, EM is guaranteed to converge.

DBSCAN

Both k-means and GMMs struggle with **non-spherical clusters**. A cluster shaped like a crescent or ring cannot be captured by any number of Gaussian components. **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** takes a different approach: it defines clusters as **dense regions of points separated by sparse regions**. This allows it to find arbitrarily shaped clusters.

Core Concepts

DBSCAN has **two hyperparameters**: ϵ (epsilon) - a **distance threshold** and MinPts - a **minimum number of neighbours**. With these, points are classified into three types:

- **Core point**

A point with **at least MinPts** other points within **distance ϵ** .

Core points make up the interior of dense regions.

- **Border point**

A point **within distance ϵ** of a core point, but with **fewer than MinPts neighbours itself**. These are found on the edge of clusters.

- **Noise Point**

A point that is neither core nor border. These are outliers - isolated points in sparse regions.

A cluster is a **maximal set of core points** that are **density-connected**, reachable from each other via chains of core points within distance ϵ .

The Algorithm

- For each unvisited point, compute its **ϵ -neighbourhood** (all points within distance ϵ).
- If fewer than MinPts neighbours: mark as noise (tentatively - it may later become a border point).
- If at least MinPts neighbours: start a new cluster. Add all ϵ -neighbours to the cluster. For each neighbour that is **also a core point**, recursively add their neighbours to the cluster.
- Continue until no more points can be added to the current cluster. Move to the next unvisited point.

Properties and Limitations

- **Does not require K**

The number of clusters is determined automatically from the data density.

- **Handles noise**

Noise points are explicitly identified and not forced into clusters.

- **Arbitrary Shapes**

Clusters can take any shape, not just spherical or elliptical.

- **Struggles with varying density**

A single ϵ cannot simultaneously capture both dense and sparse clusters.

- **Sensitive to hyperparameters**

ϵ and MinPts must be tuned carefully.

